Reparto de tareas — PetCare: Home → Mis Mascotas → Detalle → Registrar Cuidado (MVVM + Room + Repositorio + DI, sin NavHost/NavController)

Contexto

El repo `PetCare` es, hoy, el proyecto Android por defecto generado por Android Studio: solo existe `MainActivity.kt` y el tema de Compose (`ui/theme/`). No hay pantallas, no hay Room, no hay Navigation Compose, no hay ViewModels — nada específico de PetCare todavía. El diseño (wireframes, requisitos funcionales, fichas de persona) ya está definido en el documento de la Etapa 1 (Trabajo Grupal 1).

Esta versión del plan es una auditoría + corrección de la anterior: se revisó contra **los dos documentos del profesor** (Etapa 1 y Etapa 2) y contra **todo lo que se ha pedido en el chat**, y se cerraron los vacíos encontrados. Sigue **sin NavHost/NavController** (restringido). Lo nuevo respecto a la versión anterior:

1. **Segunda tabla maestra + tabla transaccional** (el vacío más importante — el documento de la Etapa 1 pide "mínimo 2 tablas maestras y 1 tabla transaccional", y antes solo teníamos `Mascota`).
2. **CRUD completo de Cuidado** (RF05–RF09 del documento de la Etapa 1: registrar, consultar, editar, eliminar cuidados, y elegir el tipo).
3. **Eventos puntuales con `Channel`/`Flow`** (lo pedía el documento de la Etapa 2 y antes no aplicaba porque no había ninguna escritura; ahora que hay un formulario de Cuidado, sí aplica).
4. **Material Design 3 explícito** en cada pantalla (antes quedaba implícito).
5. **Extras opcionales listados** (Ícono adaptativo, WindowSizeClass) para que quede claro que existen pero no son obligatorios.
6. **Recordatorio de la presentación de cierre** (elevator pitch de 5 minutos).

Supuestos que hice para cerrar estos vacíos (avísame si alguno no es lo que quieres)

- **Interpretación de "2 tablas maestras + 1 transaccional"**: `Mascota` (maestra, ya existía) + `TipoCuidado` (maestra, nueva — catálogo fijo: Vacunación, Baño, Desparasitación, Visita veterinaria) + `Cuidado` (transaccional, nueva — un registro por cada vez que se hizo un cuidado, con fecha y notas, ligado a una mascota y a un tipo).
- `Mascota` **sigue siendo de solo lectura** (no se agregan pantallas de registrar/editar/eliminar mascota en esta entrega — sigue precargada con Rocky). El CRUD completo que sí se agrega es sobre `Cuidado`, porque es lo que pide explícitamente el documento de la Etapa 1 (RF05–RF08) y es lo mínimo para tener una tabla transaccional real.
- **Una sola pantalla nueva** (`RegistrarCuidadoScreen`) sirve para **crear y editar** un cuidado (según si le llega un id o no), y desde ahí mismo se puede eliminar. Así no se agregan 3-4 pantallas nuevas, solo 1.
- La cadena de navegación crece un nivel más: **Home** → **Mascotas** → **Detalle** → **Registrar/Editar Cuidado**. Sigue siendo una cadena lineal (no un grafo), así que entendemos que sigue dentro de la restricción del profesor de "padre → hijo" aplicada varias veces, pero avísame si tu profesor la quiere más corta.

Cómo esto encaja con la rúbrica de la Etapa 2 (actualizado)

Criterio de la rúbrica	¿Se cubre en esta entrega?
1. Capa de UI & State Hoisting (Compose, Material 3)	Sí — `Route` (stateful) / `Screen` (stateless), sin pasar ViewModel a hijos, `collectAsStateWithLifecycle()`, componentes Material 3 explícitos (ver Paso 1).
2. Arquitectura MVVM & Gestión de Estado Reactivo	Sí — un ViewModel por pantalla, `StateFlow<UiState>` sellado, y ahora también **eventos puntuales con** `Channel` en el formulario de Cuidado.
3. Capa de Datos (Room & Single Source of Truth)	Sí — 3 entidades (2 maestras + 1 transaccional), lecturas con `Flow<T>`, escrituras `suspend fun`, CRUD completo sobre `Cuidado`.
4. Patrón Repositorio y Desacoplamiento	Sí — `MascotaRepository` + `CuidadoRepository`, cada uno con su interfaz + implementación.
5. Navegación e Inyección de Dependencias (DI)	Solo la mitad: la DI (`AppContainer` + fábricas) sí se cubre. La navegación con `NavHost`/`NavController` sigue pendiente para cuando el profesor la habilite.
Extra: Ícono adaptativo / WindowSizeClass / UseCases	No incluidos — son opcionales, ver "Fuera de alcance".

Estructura de paquetes a crear

```
app/src/main/java/com/am/septimob/petcare/
■■■ data/
■   ■■■ Mascota.kt                    (@Entity — tabla maestra 1)
■   ■■■ TipoCuidado.kt                (@Entity — tabla maestra 2, catálogo fijo)
■   ■■■ Cuidado.kt                    (@Entity — tabla transaccional, FK a Mascota y TipoCuidado)
■   ■■■ MascotaDao.kt
■   ■■■ TipoCuidadoDao.kt
■   ■■■ CuidadoDao.kt                 (incluye insertar/actualizar/eliminar = CRUD completo)
■   ■■■ PetCareDatabase.kt            (@Database con las 3 entidades + precarga inicial)
■   ■■■ MascotaRepository.kt + MascotaRepositoryImpl.kt
■   ■■■ CuidadoRepository.kt + CuidadoRepositoryImpl.kt   (maneja Cuidado y TipoCuidado)
■■■ AppContainer.kt                   (ahora expone mascotaRepository y cuidadoRepository)
■■■ Pantalla.kt                       (enum: HOME, MASCOTAS, DETALLE, REGISTRAR_CUIDADO)
■■■ PetCareApp.kt                     (Persona D — contenedor visual, arma los ViewModels vía factory)
■■■ HomeScreen.kt + HomeViewModel.kt                          (Persona A)
■■■ MascotasScreen.kt + MascotasViewModel.kt                  (Persona B)
■■■ DetalleMascotaScreen.kt + DetalleMascotaViewModel.kt      (Persona C — ahora incluye la lista de cu
■■■ RegistrarCuidadoScreen.kt + RegistrarCuidadoViewModel.kt  (Persona C — nueva, crear/editar/eliminar
```

Nota sobre el estilo de código: seguimos con **paquete plano** (nada de `ui/screens/...`) y `@Preview` por pantalla, que es lo que se veía en los codelabs de Afirmaciones/Woof/Tip Calculator que el equipo ya conoce — pero hay que ser honestos: **Room, ViewModel, Repositorio y Channel son contenido nuevo que esos codelabs no cubren en absoluto** (son apps de una sola pantalla sin base de datos). Lo único que se mantiene "al estilo de esas clases" es la organización de archivos y la forma de los composables, no la

arquitectura de datos, que es enteramente nueva y viene del documento de la Etapa 2.

Paso 0 — Base compartida (sesión conjunta del equipo)

Corrección sobre el reparto que había planteado antes: dije que cada sub-hilo podía hacer "su entidad + su Dao + su Repositorio" de punta a punta, pero eso no es cierto una vez que se mira la dependencia real: `Cuidado` tiene una llave foránea hacia `Mascota` y hacia `TipoCuidado`, y la interfaz `CuidadoRepository` usa los tipos `Cuidado` y `TipoCuidado` en su firma. Es decir, **las 3 entidades + las 2 interfaces de Repositorio están enlazadas entre sí** y tienen que salir como un solo bloque compartido, rápido, antes de repartir nada. Lo que sí se puede paralelizar de verdad es la parte de *implementación* (Dao + RepositoryImpl) de cada familia, una vez que ese bloque ya existe.

Por eso el orden queda así:

0.1 — Dependencias en Gradle (sin depender de nada, se puede hacer en paralelo con todo lo demás)

- Plugin **KSP** (`com.google.devtools.ksp`).
- `androidx.room:room-runtime`, `androidx.room:room-ktx`, `androidx.room:room-compiler` (con `ksp(...)`).
- `androidx.lifecycle:lifecycle-viewmodel-compose`.

Usar la versión estable más reciente compatible con Kotlin `2.2.10` / Compose BOM `2026.02.01` — revisar en Android Studio, no copiar un número a ciegas.

0.2 — `Pantalla.kt` (tampoco depende de nada — antes quedaba metido sin razón entre los pasos de datos)

```
enum class Pantalla { HOME, MASCOTAS, DETALLE, REGISTRAR_CUIDADO }
```

0.3 — Contrato de datos (bloqueante: esto es lo primero que hay que subir, porque todo lo demás depende de esto)

```
// data/Mascota.kt
@Entity(tableName = "mascotas")
data class Mascota(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val nombre: String,
    val especie: String,
    val raza: String,
    val fechaNacimiento: String,
    val sexo: String,
    val pesoKg: Double
)

// data/TipoCuidado.kt — tabla maestra 2
@Entity(tableName = "tipos_cuidado")
data class TipoCuidado(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val nombre: String // "Vacunación", "Baño", "Desparasitación", "Visita veterinaria"
)

// data/Cuidado.kt — tabla transaccional (por eso necesita que Mascota y TipoCuidado ya existan)
@Entity(
    tableName = "cuidados",
    foreignKeys = [
        ForeignKey(entity = Mascota::class, parentColumns = ["id"], childColumns = ["mascotaId"], onDele
```

```
        ForeignKey(entity = TipoCuidado::class, parentColumns = ["id"], childColumns = ["tipoCuidadoId"]
    ],
    indices = [Index("mascotaId"), Index("tipoCuidadoId")]
)
data class Cuidado(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val mascotaId: Int,
    val tipoCuidadoId: Int,
    val fecha: String,
    val notas: String
)
```

```
// data/MascotaRepository.kt
interface MascotaRepository {
    fun obtenerMascotas(): Flow<List<Mascota>>
    fun obtenerMascotaPorId(id: Int): Flow<Mascota?>
}
```

```
// data/CuidadoRepository.kt — usa Cuidado y TipoCuidado en su firma, por eso va después de 0.3
data class CuidadoConTipo(
    val id: Int,
    val fecha: String,
    val notas: String,
    val tipoCuidadoNombre: String
)

interface CuidadoRepository {
    fun obtenerCuidadosDeMascota(mascotaId: Int): Flow<List<CuidadoConTipo>>
    fun obtenerCuidadoPorId(id: Int): Flow<Cuidado?>
    fun obtenerTiposDeCuidado(): Flow<List<TipoCuidado>>
    suspend fun guardar(cuidado: Cuidado): Long
    suspend fun actualizar(cuidado: Cuidado)
    suspend fun eliminar(cuidado: Cuidado)
}
```

En cuanto existe este bloque completo (3 entidades + 2 interfaces), **las personas A, B y C ya pueden escribir sus ViewModels** sin esperar a que Room esté implementado de verdad, y **los 2 sub-hilos de abajo ya pueden arrancar en paralelo**.

0.4 — Sub-hilo 1: implementación de Mascota (independiente del sub-hilo 2)

```
// data/MascotaDao.kt
@Dao
interface MascotaDao {
    @Query("SELECT * FROM mascotas")
    fun obtenerTodas(): Flow<List<Mascota>>

    @Query("SELECT * FROM mascotas WHERE id = :id")
    fun obtenerPorId(id: Int): Flow<Mascota?>

    @Insert
    suspend fun insertar(mascota: Mascota)
}
```

```
// data/MascotaRepositoryImpl.kt
class MascotaRepositoryImpl(private val dao: MascotaDao) : MascotaRepository {
    override fun obtenerMascotas(): Flow<List<Mascota>> = dao.obtenerTodas()
    override fun obtenerMascotaPorId(id: Int): Flow<Mascota?> = dao.obtenerPorId(id)
}
```

0.5 — Sub-hilo 2: implementación de TipoCuidado + Cuidado (independiente del sub-hilo 1)

```
// data/TipoCuidadoDao.kt
```

```
@Dao
interface TipoCuidadoDao {
    @Query("SELECT * FROM tipos_cuidado")
    fun obtenerTodos(): Flow<List<TipoCuidado>>

    @Insert
    suspend fun insertar(tipo: TipoCuidado)
}
```

```
// data/CuidadoDao.kt — CRUD completo (RF05-RF08)
@Dao
interface CuidadoDao {
    @Query("""
        SELECT cuidados.id AS id, cuidados.fecha AS fecha, cuidados.notas AS notas,
               tipos_cuidado.nombre AS tipoCuidadoNombre
        FROM cuidados
        INNER JOIN tipos_cuidado ON cuidados.tipoCuidadoId = tipos_cuidado.id
        WHERE cuidados.mascotaId = :mascotaId
        ORDER BY cuidados.fecha DESC
    """)
    fun obtenerCuidadosDeMascota(mascotaId: Int): Flow<List<CuidadoConTipo>>

    @Query("SELECT * FROM cuidados WHERE id = :id")
    fun obtenerPorId(id: Int): Flow<Cuidado?>

    @Insert
    suspend fun insertar(cuidado: Cuidado): Long

    @Update
    suspend fun actualizar(cuidado: Cuidado)

    @Delete
    suspend fun eliminar(cuidado: Cuidado)
}
```

```
// data/CuidadoRepositoryImpl.kt
class CuidadoRepositoryImpl(
    private val cuidadoDao: CuidadoDao,
    private val tipoCuidadoDao: TipoCuidadoDao
) : CuidadoRepository {
    override fun obtenerCuidadosDeMascota(mascotaId: Int) = cuidadoDao.obtenerCuidadosDeMascota(mascotaI
    override fun obtenerCuidadoPorId(id: Int) = cuidadoDao.obtenerPorId(id)
    override fun obtenerTiposDeCuidado() = tipoCuidadoDao.obtenerTodos()
    override suspend fun guardar(cuidado: Cuidado) = cuidadoDao.insertar(cuidado)
    override suspend fun actualizar(cuidado: Cuidado) = cuidadoDao.actualizar(cuidado)
    override suspend fun eliminar(cuidado: Cuidado) = cuidadoDao.eliminar(cuidado)
}
```

0.6 — Unión de los 2 sub-hilos (la hace quien lidera Persona D, una vez que 0.4 y 0.5 están listos)

```
// data/PetCareDatabase.kt — junta las 3 entidades y los 3 Dao de ambos sub-hilos
@Database(entities = [Mascota::class, TipoCuidado::class, Cuidado::class], version = 1, exportSchema = f
abstract class PetCareDatabase : RoomDatabase() {
    abstract fun mascotaDao(): MascotaDao
    abstract fun tipoCuidadoDao(): TipoCuidadoDao
    abstract fun cuidadoDao(): CuidadoDao

    companion object {
        @Volatile private var INSTANCE: PetCareDatabase? = null

        fun getInstance(context: Context): PetCareDatabase =
```

```
                INSTANCE ?: synchronized(this) {
                    Room.databaseBuilder(context, PetCareDatabase::class.java, "petcare.db")
                        .addCallback(object : RoomDatabase.Callback() {
                            override fun onCreate(db: SupportSQLiteDatabase) {
                                super.onCreate(db)
                                // Precarga: sin pantallas de "Registrar mascota"/"Registrar tipo" todavía,
                                // así que se siembra Rocky y el catálogo fijo de tipos de cuidado.
                                CoroutineScope(Dispatchers.IO).launch {
                                    val instancia = getInstance(context)
                                    instancia.mascotaDao().insertar(
                                        Mascota(nombre = "Rocky", especie = "Perro", raza = "Mestizo",
                                            fechaNacimiento = "12/03/2021", sexo = "Macho", pesoKg = 18.5)
                                    )
                                    listOf("Vacunación", "Baño", "Desparasitación", "Visita veterinaria").fo
                                        instancia.tipoCuidadoDao().insertar(TipoCuidado(nombre = it))
                                    }
                                }
                            }
                        })
                        .build()
                        .also { INSTANCE = it }
                }
            }
        }

        // AppContainer.kt — junta los 2 RepositoryImpl de ambos sub-hilos
        class AppContainer(context: Context) {
            private val database = PetCareDatabase.getInstance(context)
            val mascotaRepository: MascotaRepository = MascotaRepositoryImpl(database.mascotaDao())
            val cuidadoRepository: CuidadoRepository =
                CuidadoRepositoryImpl(database.cuidadoDao(), database.tipoCuidadoDao())
        }
```

`MainActivity.kt` sigue igual que en la versión anterior: crea `AppContainer` una vez y se lo pasa a `PetCareApp`.

Paso 1 — Trabajo en paralelo (una persona por pantalla; Persona C ahora tiene 2 pantallas)

Regla estricta: el composable `XRoute` es el único que conoce el ViewModel. El composable `XScreen` solo recibe el `UiState` + lambdas. **Usar componentes Material 3** en todas las pantallas: `Scaffold` + `TopAppBar`, `Card`, `Button`/`FilledTonalButton`, `OutlinedTextField`, `CircularProgressIndicator` para `Loading`, y colores desde `MaterialTheme.colorScheme` (no colores sueltos hardcodeados).

Persona A — `HomeScreen.kt` + `HomeViewModel.kt` (sin cambios respecto a la versión anterior)

`HomeUiState` (`Loading`/`Success(cantidadMascotas)`/`Error`), `HomeRoute` colecta el `StateFlow`, `HomeScreen` dibuja con `Scaffold`/`TopAppBar`/`Card`. Wireframe "Pantalla 2 — Home".

Persona B — `MascotasScreen.kt` + `MascotasViewModel.kt` (sin cambios respecto a la versión anterior)

`MascotasUiState` (`Loading`/`Success(mascotas)`/`Error`), `LazyColumn` de `Card` por mascota. Wireframe "Pantalla 3 — Mis mascotas".

Persona C — `DetalleMascotaScreen.kt` + `DetalleMascotaViewModel.kt` (ampliada) + `RegistrarCuidadoScreen.kt` + `RegistrarCuidadoViewModel.kt` (nueva)

Esta persona tiene ahora el trabajo más grande de las 3 pantallas — si Persona A o B terminan antes, pueden ayudar aquí.

`DetalleMascotaViewModel` ahora combina dos flujos (datos de la mascota + su lista de cuidados):

```
sealed interface DetalleUiState {
    data object Loading : DetalleUiState
    data class Success(val mascota: Mascota, val cuidados: List<CuidadoConTipo>) : DetalleUiState
    data class Error(val mensaje: String) : DetalleUiState
}

class DetalleMascotaViewModel(
    mascotaId: Int,
    mascotaRepository: MascotaRepository,
    cuidadoRepository: CuidadoRepository
) : ViewModel() {
    val uiState: StateFlow<DetalleUiState> =
        combine(
            mascotaRepository.obtenerMascotaPorId(mascotaId),
            cuidadoRepository.obtenerCuidadosDeMascota(mascotaId)
        ) { mascota, cuidados ->
            if (mascota != null) DetalleUiState.Success(mascota, cuidados)
            else DetalleUiState.Error("Mascota no encontrada")
        }
            .catch { emit(DetalleUiState.Error(it.message ?: "Error al cargar")) }
            .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5_000), DetalleUiState.Loading)
}
```

`DetalleMascotaScreen` dibuja los datos de la mascota (wireframe "Pantalla 5 — Detalle de mascota") **más** una lista de `cuidados` (estilo wireframe "Pantalla 6 — Historial de cuidados") y un botón "Registrar cuidado". Cada ítem de cuidado es clickeable → `onCuidadoClick(cuidadoId)` (para editar).

`RegistrarCuidadoViewModel` — sirve para crear (sin `cuidadoId`) o editar (con `cuidadoId`), y usa un `Channel` para los eventos puntuales que pide la Etapa 2:

```
sealed interface RegistrarCuidadoUiState {
    data object Cargando : RegistrarCuidadoUiState
    data class Formulario(
        val tiposCuidado: List<TipoCuidado>,
        val tipoCuidadoSeleccionadoId: Int? = null,
        val fecha: String = "",
        val notas: String = "",
        val esEdicion: Boolean = false
    ) : RegistrarCuidadoUiState
    data class Error(val mensaje: String) : RegistrarCuidadoUiState
}

sealed interface RegistrarCuidadoEvento {
    data object GuardadoExitoso : RegistrarCuidadoEvento
    data object EliminadoExitoso : RegistrarCuidadoEvento
    data class Error(val mensaje: String) : RegistrarCuidadoEvento
}

class RegistrarCuidadoViewModel(
    private val mascotaId: Int,
    private val cuidadoId: Int?,
    private val repository: CuidadoRepository
) : ViewModel() {
    private val _eventos = Channel<RegistrarCuidadoEvento>()
    val eventos = _eventos.receiveAsFlow()
    // uiState: carga tiposCuidado (y el cuidado existente si cuidadoId != null) y arma Formulario
```

```
    fun guardar(tipoCuidadoId: Int, fecha: String, notas: String) {
        viewModelScope.launch {
            try {
                repository.guardar(Cuidado(id = cuidadoId ?: 0, mascotaId = mascotaId,
                    tipoCuidadoId = tipoCuidadoId, fecha = fecha, notas = notas))
                _eventos.send(RegistrarCuidadoEvento.GuardadoExitoso)
            } catch (e: Exception) {
                _eventos.send(RegistrarCuidadoEvento.Error(e.message ?: "No se pudo guardar"))
            }
        }
    }

    fun eliminar() {
        viewModelScope.launch {
            // repository.eliminar(...) del cuidado actual, luego:
            _eventos.send(RegistrarCuidadoEvento.EliminadoExitoso)
        }
    }
}
```

`RegistrarCuidadoRoute` colecta `uiState` **y** escucha `eventos` con `LaunchedEffect`, para mostrar un Snackbar/Toast y volver a Detalle cuando llega `GuardadoExitoso`/`EliminadoExitoso` — esto es justo el patrón de "eventos puntuales con canales/flujos" que pide la rúbrica de la Etapa 2. `RegistrarCuidadoScreen` usa `OutlinedTextField` para fecha/notas y un `ExposedDropdownMenuBox` (Material 3) para elegir el tipo de cuidado. Wireframe "Pantalla 7 — Registrar/editar cuidado" + "Pantalla 8 — Detalle del cuidado" (para el botón eliminar).

Persona D — `PetCareApp.kt` (contenedor, arma los ViewModels vía factory, navegación manual con `Pantalla`)

```
@Composable
fun PetCareApp(appContainer: AppContainer) {
    var pantallaActual by remember { mutableStateOf(Pantalla.HOME) }
    var mascotaIdSeleccionada by remember { mutableStateOf<Int?>(null) }
    var cuidadoIdSeleccionado by remember { mutableStateOf<Int?>(null) }

    when (pantallaActual) {
        Pantalla.HOME -> {
            val factory = viewModelFactory { initializer { HomeViewModel(appContainer.mascotaRepository)
            HomeRoute(viewModel = viewModel(factory = factory),
                onVerMascotasClick = { pantallaActual = Pantalla.MASCOTAS })
        }

        Pantalla.MASCOTAS -> {
            val factory = viewModelFactory { initializer { MascotasViewModel(appContainer.mascotaReposit
            MascotasRoute(
                viewModel = viewModel(factory = factory),
                onMascotaClick = { id -> mascotaIdSeleccionada = id; pantallaActual = Pantalla.DETALLE }
                onBackClick = { pantallaActual = Pantalla.HOME }
            )
        }

        Pantalla.DETALLE -> {
            val id = mascotaIdSeleccionada
            if (id != null) {
                val factory = viewModelFactory {
                    initializer { DetalleMascotaViewModel(id, appContainer.mascotaRepository, appContair
                }
                DetalleMascotaRoute(
                    viewModel = viewModel(factory = factory),
```

```
                        onRegistrarCuidadoClick = { cuidadoIdSeleccionado = null; pantallaActual = Pantalla.
                        onCuidadoClick = { cuidadoId -> cuidadoIdSeleccionado = cuidadoId; pantallaActual =
                        onBackClick = { pantallaActual = Pantalla.MASCOTAS }
                    )
                }
            }

            Pantalla.REGISTRAR_CUIDADO -> {
                val mascotaId = mascotaIdSeleccionada
                if (mascotaId != null) {
                    val factory = viewModelFactory {
                        initializer { RegistrarCuidadoViewModel(mascotaId, cuidadoIdSeleccionado, appContain
                    }
                    RegistrarCuidadoRoute(
                        viewModel = viewModel(factory = factory),
                        onListo = { pantallaActual = Pantalla.DETALLE }
                    )
                }
            }
        }
    }
```

Notas para esta persona:

- El `viewModelFactory { initializer { ... } }` sigue siendo un `ViewModelProvider.Factory` válido.
- Sigue sin `NavHost`, así que el botón físico/gesto de "atrás" del sistema sigue sin retroceder solo entre pantallas — mismo caveat de siempre.
- Ahora coordina la integración de **2 repositorios** (`mascotaRepository`, `cuidadoRepository`), no solo uno.

Paso 2 — Integración y verificación (todo el equipo)

1. Cada persona sube su(s) archivo(s) a su propia rama y abre PR contra `main`.
2. Se integra en orden: `data/` completo (las 3 entidades + los 2 repositorios) + `AppContainer` + `Pantalla`, luego las pantallas (Home, Mascotas, Detalle, RegistrarCuidado), luego `PetCareApp.kt` + `MainActivity.kt`.
3. Compilar y correr. Probar a mano el **comportamiento offline-first** (lo van a pedir en la demo en vivo de la presentación de cierre):

- Abrir la app: Home muestra "1" mascota.
- Ir a Detalle de Rocky, tocar "Registrar cuidado", elegir un tipo (p. ej. "Baño"), poner fecha y notas, guardar.
- Verificar que aparece de inmediato en la lista de cuidados de Rocky (sin recargar nada a mano — eso es la reactividad de `Flow`).
- Tocar ese cuidado para editarlo, cambiar la nota, guardar — se actualiza en la lista.
- Eliminarlo — desaparece de la lista.
- **Cerrar la app por completo (matar el proceso), activar modo avión, y volver a abrirla**: todo lo que quedó guardado (Rocky y los cuidados que no borraste) debe seguir ahí — es la prueba explícita que pide el documento de la Etapa 2 para la presentación.

1. Revisar que ningún `XScreen` reciba el ViewModel ni el repositorio directamente.

2. Revisar que ningún ViewModel cree `PetCareDatabase`, `MascotaRepositoryImpl` o `CuidadoRepositoryImpl` directamente — siempre a través de `AppContainer`.

Recordatorio: presentación de cierre (elevator pitch, 5 minutos)

Esto no es una tarea de código, pero el documento de la Etapa 2 la exige junto con la entrega, así que no hay que olvidarla al planear los tiempos del equipo:

- **1 min** — nombre de la app y el problema que resuelve.
- **2 min** — demo en vivo: registrar/editar/eliminar un cuidado de Rocky, y la prueba de offline-first (cerrar app + modo avión + reabrir).
- **2 min** — arquitectura: mostrar el `Route`/`Screen` sin pasar el ViewModel, el `StateFlow`/`Flow` desde Room hasta la UI, y cómo se entregan el repositorio y el ViewModel sin instanciarlos a mano (`AppContainer`).
- Los 4 integrantes deben poder responder preguntas de cualquier capa (esto lo dice el documento explícitamente), así que conviene que cada quien entienda al menos su propia capa a fondo antes del día de la presentación.

Fuera de alcance de esta entrega (a propósito)

- **NavHost/NavController de Navigation Compose** — se usará cuando el profesor lo habilite. La navegación sigue siendo manual, con `Pantalla` + `when` en el contenedor.
- **Registrar/editar/eliminar Mascota** — sigue de solo lectura, precargada con Rocky. El CRUD completo que se agregó en esta versión es sobre `Cuidado`, no sobre `Mascota`.
- DataStore, Splash Screen, pantalla de Configuración — siguen pendientes.
- `Hilt` como alternativa a `AppContainer` — se usa DI manual por ahora.
- Casos de uso (`UseCases`) de la capa de dominio — es un extra opcional del documento (+0.3 pts) y por ahora la lógica de cada ViewModel es simple, no hay validaciones/transformaciones complejas que justifiquen esa capa todavía.
- Ícono adaptativo vectorizado (+0.3 pts) y diseño responsivo con `WindowSizeClass` (+0.4 pts) — son extras opcionales del documento, no obligatorios; se pueden agregar después si sobra tiempo.
- Manejo del botón físico/gesto de "atrás" del sistema entre pantallas.